NoCDAS: A Cycle-Accurate NoC-Based Deep Neural Network Accelerator Simulator

WENYAO ZHU, Division of Electronics and Embedded Systems, KTH Royal Institute of Technology, Stockholm, Sweden
YIZHI CHEN, Division of Electronics and Embedded Systems, KTH Royal Institute of Technology, Stockholm, Sweden
ZHONGHAI LU*, Division of Electronics and Embedded Systems, KTH Royal Institute of Technology, Stockholm, Sweden

Network-on-Chip (NoC) has been widely adopted for Deep Neural Network (DNN) accelerator designs to solve the data communication problem for the large-scale processing element array. As the complexity of these DNN accelerators grows significantly, effective design-space exploration before hardware prototyping becomes crucial. However, the existing simulation tools for NoC-based DNN accelerators are limited in providing accurate hardware microarchitecture representations and execution time analysis. In this paper, we present a cycle-accurate simulator for NoC-based DNN accelerators called NoCDAS. The proposed NoCDAS can accurately simulate DNN computation flow on NoC hardware and the correctness of inference output is validated. In addition, NoCDAS supports highly flexible NoC hardware definitions to quantify the end-to-end latency, which allows efficient evaluation of different NoC-based DNN accelerator design parameters. We showcase this ability by running three DNN models on the proposed NoCDAS with various configurations encompassing NoC size, mapping strategy, and core placement.

CCS Concepts: • **Computing methodologies** → **Simulation tools**; • **Computer systems organization** → **Neural networks**; **Interconnection architectures**; • **Hardware** → **Hardware accelerators**.

Additional Key Words and Phrases: Network-on-Chip, Neural network accelerator, NoC-based DNN, Cycle-accurate simulation, Design parameter exploration

1 Introduction

The rapid evolution of deep learning with neural networks has greatly improved the state-of-the-art in many real-world artificial intelligence (AI) applications such as visual pattern recognition, object detection, and natural language processing [15, 22, 36]. However, the quest for heightened accuracy in Deep Neural Network (DNN) models necessitates deeper and more complex architectures, leading to increased computation costs. Moreover, the growth in the demand for AI on embedded edge devices poses a significant challenge to their practical deployment in daily life.

DNNs typically comprise millions of neurons, each executing a vast number of multiply-and-accumulation (MAC) operations, making efficient computation crucial for performance. As a result, hardware accelerators that

*Corresponding author: Zhonghai Lu, zhonghai@kth.se

Authors' Contact Information: Wenyao Zhu, Division of Electronics and Embedded Systems, KTH Royal Institute of Technology, Stockholm, Stockholm, Sweden; e-mail: wenyao@kth.se; Yizhi Chen, Division of Electronics and Embedded Systems, KTH Royal Institute of Technology, Stockholm, Stockholm, Sweden; e-mail: yizhic@kth.se; Zhonghai Lu, Division of Electronics and Embedded Systems, KTH Royal Institute of Technology, Stockholm, Stockholm, Sweden; e-mail: zhonghai@kth.se.

ACM 1558-1195/2025/4-ART
https://doi.org/10.1145/3729169

enable parallel processing are a practical solution for efficient DNN inference [34]. Recent ASIC accelerators for DNN [10, 14, 40] adopt large-scale processing element (PE) arrays to enable concurrent execution. They can achieve optimal performance and energy efficiency using optimization techniques such as data reuse and bit-width adaptive computing. However, their use is often limited to particular types and shapes of DNN layers because of their rigidly designed data paths. These accelerators cannot be reconfigured at runtime to adapt to new DNN tasks [5]. Given that real DNN workloads can differ from those anticipated during design, the demand for better computational flexibility has grown.

The Network-on-Chip (NoC) based DNN accelerator (NoC-DNN) is a suitable alternative to conventional accelerator designs. NoC arranges links and routers to connect numerous PEs and allows arbitrary communication between cores. Data transmission occurs in the form of packets, which are independent of the computation process in PE. NoC-DNN efficiently supports flexible traffic patterns for various DNN data flows, thereby enhancing computational flexibility at the cost of energy efficiency compared to PE array-based accelerators. Several NoC-DNNs have been designed [11, 21, 25, 32, 39] that can support a wide range of DNNs on a single chip.

To efficiently explore the NoC-DNN design space and reduce the hardware design time and cost, it is necessary to utilize a NoC-DNN hardware simulator. Chen *et al.* developed the first NoC-based neural network simulator NN-Noxim [8] for Multilayer Perceptron and extended it to support Convolutional Neural Networks in [9]. However, these simulators assume that the DNN model is mapped to the NoC platform at once and all weights are stored in local PEs. This is not practical for real hardware implementation. Later on, Munoz *et al.* proposed STONNE [26] as a highly modular simulation framework for end-to-end evaluation of flexible DNN accelerator architectures. They assume a more realistic tree-based NoC topology, but the supported dataflow is also limited to their pre-defined network fabrics, and detailed traffic patterns in NoC are ignored.

To the best of our knowledge, there is still no realistic microarchitecture level, cycle-accurate, and open-source NoC-DNN simulator that supports extensive design space exploration for NoC over various DNN models. In this case, we propose an open-source cycle-accurate NoC-based DNN accelerator simulator, called NoCDAS, that can be applied for end-to-end DNN inference performance evaluation on the NoC-DNN platform. It is a time-stepped simulator implemented in C++14 using standard template libraries. We have made our simulator open access (https://github.com/CRDloghorizon/NoCDAS). We summarize our contributions as follows:

- We develop a NoC-based DNN accelerator simulator that supports cycle-accurate discrete event simulation for NoC-DNN evaluation with flexible NoC configurations.
- We compare the features of state-of-the-art simulators for the NoC-based DNN accelerator and provide a more detailed and realistic simulation framework and processing flow to mitigate their limitation.
- We evaluate our simulator on different DNN models to validate the inference results and latency measurement. We demonstrate the exploration of NoC design parameters in the NoC-based DNN accelerator and discuss their trade-offs on hardware usage and latency.

We discuss the related work in Section 2. Then we compare different simulator designs and present our simulator framework in Section 3. Subsequently, we describe the simulation workflow and evaluation results in Section 4 and 5, respectively. We conclude in Section 6.

2 Related work

2.1 NoC-based DNN accelerators

The scalability and parallelism of NoC make it a proper infrastructure for supporting low latency and parallel communications in hardware acceleration of DNNs on many-core chips [27]. Theocharide *et al.* develop the first generic NoC-based architecture for artificial neural network (ANN) acceleration [35]. They find that NoC outperforms other architectures, such as systolic arrays and multiprocessor platforms, in terms of expandability and reconfigurability.

In recent years, the growing demand for running complex DNNs on edge devices has necessitated faster and more efficient accelerators. Researchers have put great effort into NoC-DNN architecture designs to optimize energy efficiency and execution speed. A neural network-aware mapping algorithm is developed by Liu *et al.* to minimize hop counts and balance data traffic across a mesh NoC-DNN architecture, significantly reducing routing distance [25]. Additionally, they implement a multicast transmission scheme to further lower data latency and energy consumption. Chen *et al.* extend the well-known Eyeriss accelerator [11] for running compact and sparse DNNs. They introduce a highly flexible hierarchical 2D mesh NoC that can be adapted to different amounts of data reuse to improve PE utilization. In [42], a sparsified parallelization method is developed to reduce the communication overhead for mesh-based NoC-DNN. Zhu *et al.* present a NoC-DNN with the sparse skipping technique to reduce memory access overhead and improve energy efficiency [41].

Other researchers focus on optimizing computational flexibility. MAERI [21] is built with tree-based NoC topology to support various DNN partitions and mappings. It can efficiently handle the convolutional, pooling, and fully connected layers via tiny configurable switches. Simba [32] presents a multi-chip-module approach for building large-scale DNN accelerators with two-level NoC. It employs several optimizations on tiling and dataflow to reduce latency. In addition, large distributed buffers are implemented to avoid weight transfer. Yang *et al.* [39] propose Venus, which uses a flexible NoC to support dynamical dataflow selection for multiple DNN applications on the same hardware. They assume all-to-all access from DRAM to tiles and use a single buffer in each tile to support inter-PE communication.

These accelerators involve sophisticated data flow and architecture design to ensure reconfigurability and efficiency, but the impact of the design parameter choices lacks study yet. Simulation tools can help architects explore the design space of NoC-DNNs, thereby reducing the design effort and cost. For example, the developers of Venus customize the analytical tool Timeloop [29] for performance comparison over other architectures.

2.2 Conventional DNN accelerator simulators

Conventional DNN accelerator simulators usually rely on abstract hardware architecture models to provide a coarse understanding of their performance characteristics when executing a wide range of DNNs.

SMAUG [38] is a DNN framework that supports end-to-end simulation for various DNN workloads on diverse accelerator modelings. However, its hardware architecture is limited to the backend simulation engine gem5-Aladdin [33], such as a systolic array or a convolution engine. So NoC-based accelerator structures can not be explored easily from SMAUG. On the other hand, since it is a trace-based simulator, it has to use a sampling approach for large DNNs to reduce the simulation time, which impedes the full model evaluation for NoC-DNN.

MAESTRO [20] is an analytical cost model that studies the data-centric mapping of DNN tasks to hardware resources. It can simulate diverse dataflow choices for a DNN model and hardware configuration and estimate the execution time and energy efficiency. The communication delay via NoC is estimated based on a pipe model involving bandwidth and average delay. Though their reported runtime is close to the RTL simulation of two DNN accelerators, the detailed NoC configuration exploration is not supported.

2.3 NoC-DNN simulators

Unlike conventional DNN accelerator simulators, NoC-DNN simulators aim to implement a more detailed and flexible NoC hardware framework. They emulate both data transmission and neuron computation behaviors to provide insights into various design choices [6].

STONNE [26] serves as a backend in high-level DNN frameworks to simulate the inference process on a flexible DNN accelerator with modeled microarchitecture. It organizes three configurable components, the distribution network, the multiplier network, and the reduction network, to model the most current DNN accelerators. Though

it supports various communication flows, its topology is limited to this three-tier architecture and lacks the support for canonical NoC interconnected PE structure.

Chen *et al.* [9] propose CNN-Noxim, a high-level cycle-accurate simulator for convolutional neural networks, and then improve it to DNNoC-sim [6] that introduces the cluster and slicing techniques for DNN models in addition to grouping. They have similar simulation workflow definitions, while DNNoC-sim supports small-sized NoC configurations as it doesn't need to map the DNN task at once.

These simulators excel in rapidly exploring optimization opportunities for DNN-NoC design but are short on comprehensively examining the impact of DNN-NoC architectures.

2.4 Level of detail in NoC-DNN simulators

A NoC simulator can be categorized by its levels of detail, which range from interface, capacity, flit, and hardware level [13]. The interface level has the least network details to investigate the general behavior of NoC. In contrast, the flit level simulates very detailed microarchitectures of the network to trace individual data packets. The capacity level stands in between, which serves for initial performance assessment and quick reconfiguration. Normally the NoC simulator will not go to the hardware level to avoid unnecessary costs of time to build a physical implementation and run a simulation.

Similarly, these simulation levels also work for the NoC-DNN simulator designs. Simulation-based analytical tools such as NNest [18] and Timeloop [29] aim for early-stage design space exploration of DNN accelerators via behavioral simulation, which enables architecture trade-off evaluation across a wide range of applications. Thus they stay on the interface level and approximately quantify the accelerator performance and energy efficiency under different hardware constraints and mapping policies. They cannot perform NoC architecture exploration for NoC-DNN due to their limited range of architecture support.

The aforementioned NoC-DNN simulators, STONNE, CNN-Noxim, and DNNoC-sim, offer more detailed microarchitecture modules compared to analytical simulation tools. These simulators ensure the timing correctness of DNN execution. On the other hand, they make abridged assumptions for NoC hardware. STONNE models the NoC-DNN components as three types of network building blocks. It simulates the behavior of each block individually and accumulates their executed cycles, but no detailed network traffic is considered. Therefore, STONNE is on the capacity level.

CNN-Noxim and DNNoC-sim employ the flit level NoC simulator Noxim [4] for accurate network behavior simulation. However, they assume that the PE can store and process a large number of neurons with a single data transfer, which simplifies the traffic in NoC for running DNN. Consequently, they stand between capacity and flit level.

These simulators cannot fully assess the exact performance of different NoC-DNNs. But they have inspired us regarding the features and workflow of simulation. Based on that, we present our cycle-accurate NoC-based DNN accelerator simulator, called NoCDAS. It is a more realistic and flexible simulation platform at the flit level.

3 NoC-DNN simulator design

3.1 NoC-DNN hardware architecture

A NoC-based DNN hardware accelerator mainly involves three components: processing cores, memory, and the on-chip network. **Fig.** 1 illustrates the NoC-DNN accelerator architecture, which is interconnected via a 2D mesh network.

NoC-based multi-core systems can employ heterogeneous cores to accommodate different tasks. In NoC-DNN, there are generally two types of cores. PE cores are used for neuron computations, which consist of the control logic, arithmetic logic units (ALUs), and local buffers. Memory controller (MC) cores control the access to the memory for all data requests in the NoC. They also manage the task mapping and resource allocation for PE

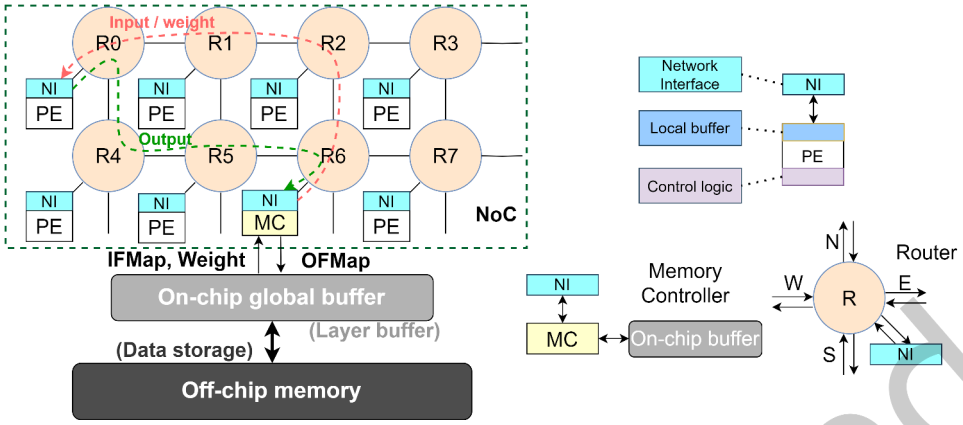

Fig. 1. NoC-based DNN accelerator architecture.

cores. The memory hierarchy for NoC-DNN is normally organized into two levels. The off-chip memory stores the weights and inputs for the whole DNN model, while the on-chip global buffer holds part of these data for fast fetch. Every processing core is connected to a NoC router through the network interface (NI). NI handles the data packetization, and the router manages the packet communication in the network.

3.2 NoC-DNN simulator features

To support a more flexible and realistic NoC hardware configuration prior to hardware level RTL prototyping as well as to maintain cycle-accurate behavior, we design the flit level NoC-DNN simulator, NoCDAS. It can track resource usage by clock cycles on a flit basis.

Since the NoC architecture for NNest, Timeloop, and STONNE are rigid, we only compare the two Noxim-based simulators [6, 9] that support flexible NoC configurations with ours. For a typical NoC-DNN simulator that accepts various DNN models, the computing task of the neural network is converted to generic neurons. Then these neurons are mapped to PEs iteratively to perform parallel computation, and NoC manages the data transmission. Though CNN-Noxim, DNNoC-sim, and our NoCDAS can all evaluate full DNN models in this manner, several features can influence the utility of the simulator. We summarize the main differences across four features in **Tab.** 1, including the PE size, task mapping, weight memory, and dataflow.

CNN-Noxim and DNNoC-sim use weight stationary computation flow, where weight memory is placed inside PE. They assume a large-size PE that can hold thousands to millions of neurons simultaneously. As a result, the local buffer should be big enough to store the corresponding inputs in one data transfer. CNN-Noxim stores all the required weights locally in PE, therefore it only involves the PE-to-PE data flow for inter-layer forwarding results transmission. Due to this setting, its task mapping is limited by the NoC size. A large DNN model can not be mapped to a small NoC when one of its layers cannot be fitted to the available PEs in the network at once. DNNoC-sim expands from CNN-Noxim with an off-chip memory module. It manages the weight update of local weight memories per mapping iteration, which assumes direct data transfer between PE and off-chip memory. The network is not involved in this update. In addition, the intermediate neuron outputs will be stored in the off-chip memory if the neuron tasks from the same layer need more than one mapping iteration. Therefore DNNoC-sim supports flexible task mapping, but its in-NoC traffic is still PE to PE.

In contrast to these simulators, our NoCDAS adopts a more realistic NoC model that involves the MC cores for managing the data request and accessing the memory. Moreover, we assume a small PE that can hold one or

Table 1. Feature comparison for NoC-DNN Simulators

	CNN-Noxim	DNNoC-sim	NoCDAS(our)
Assumed PE size (in neurons)	Large $10^3 \sim 10^6$	Large $10^3 \sim 10^6$	Small $1 \sim 10$
Task mapping	Limited by NoC size	Flexible	Flexible
Weight memory	Inside PE (All stored before simulation)	Inside PE (Update per mapping iteration)	On-chip buffer (Update per mapping iteration)
Dataflow in NoC	PE to PE	PE to PE	PE to MC MC to PE

multiple neurons at a time based on the assigned hardware resources. In this case, our simulator can also evaluate large-scale NoC-based DNN accelerators.

CNN-Noxim and DNNoC-sim use the unicast data transmission mode for PE-to-PE data flow. In recent NoC-based DNN accelerators [7, 21, 25], other data transmission modes, such as multicast and broadcast, have been used to enable data sharing across multiple PEs. These transmission modes are beneficial for convolution operations that require the same weight kernel for computation in the same output channel. Multicast and broadcast can reduce the network traffic and optimize the data transfer. However, they may also cause the scalability issue if multicast messages are broken down into multiple unicast packets in conventional NoC [1] or use specific topology such as tree-based multicast [21, 26, 28].

In our design, we adopt the unicast mode in a mesh NoC to support the data flow between PE and MC. It is possible to integrate additional multicast and broadcast transmission modes into our simulator in future versions. This can enable a more comprehensive exploration of data reuse methods and enhance the capability of NoCDAS to simulate more real-world NoC-DNN chips.

3.3 NoC-DNN simulator framework

The proposed NoC-DNN simulator is designed as a cycle-accurate simulator for NoC-based DNN hardware accelerators. **Fig.** 2 shows the overview of the simulator framework with its three major modules, the configuration input, the simulation platform, and the statistical output. The NoC-DNN simulator takes a DNN inference task as the input workload and generates corresponding results after a full model simulation.

3.3.1 Configuration input. The configuration files describe the DNN workload and the NoC parameters for simulation. Thus, two configuration files are generated as the simulator input.

The first input file is the NoC configuration, which abstracts the target NoC hardware resources. A default parameter file (parameters.hpp) is provided for all adjustable variables, including the NoC size, core configurations, and a variety of network and router parameters. Since the simulated NoC is built according to the parameter macro definitions, the NoC hardware configuration changes require the simulator's recompilation before the evaluation.

The second is the DNN configuration, which contains information about the target DNN workloads. This configuration contains three files, including the DNN model structure, the input, and the weight. The model structure is described layer-by-layer, starting with the input layer. For each layer, the layer type, input size, filter size, and non-linear activation function should be provided. In addition, if any stride or padding is added,

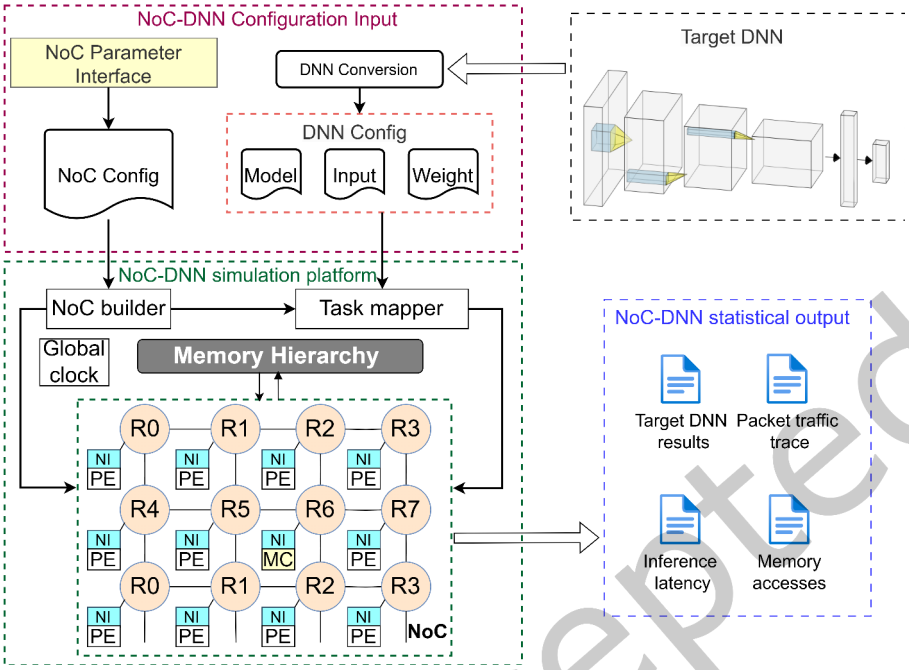

Fig. 2. Overview of the NoC-DNN simulator framework.

they should be specified in the model structure. To get accurate inference results, users need to prepare proper inputs and weights, preferably from a trained DNN model. As a lot of current DNN models are developed using open-source deep learning frameworks such as PyTorch [30], we provide a Python script to directly extract the weights from a pre-trained model that utilizes standard PyTorch APIs for DNN layers such as *nn.MaxPool2d* and *nn.Conv2d*. An example of preparing the weight and input files from a well-known CNN model LeNet-5 [23] using this script is given along with the simulator.

In addition to the full model execution (FE) mode, we provide a NoC-DNN hardware performance evaluation (RE) mode. This mode only involves the DNN model structure file as the DNN configuration and the simulator will run with random inputs and weights. Therefore, users can skip the complex training process, facilitating faster evaluation.

3.3.2 Simulation platform. The simulation platform is the core of the proposed NoCDAS, which implements the hardware architecture shown in **Fig.** 1. The simulator is driven by a global clock, which makes the simulation process clock cycle accurate. Two interfaces connect to the configuration input module for building the NoC and mapping the DNN task to the NoC.

The first interface NoC builder constructs the NoC from the NoC configuration. It defines the cores, NIs, and routers and then places them in the network according to their position. After that, it links these NoC components together with the memory hierarchy to implement the NoC-DNN hardware. Simultaneously, the second interface task mapper converts and maps the target DNN onto the established NoC by parsing the DNN configuration.

Then we initialize the off-chip memory with given inputs and weights and reset the global clock. After tasks are distributed to PEs, NoCDAS starts the simulation. At each clock cycle, it will check the NoC-DNN execution status, run the *runonestep()* function for all PEs and MCs, and update the network traffic accordingly. Since the

inter-layer data dependency exists in DNN, the simulation follows a layer-completing order to iteratively map the task of the next layer after the current layer is completed. In this case, there is no extra stall while awaiting requested data from the previous layer before they are updated in the on-chip buffer. The global clock drives the simulation and stops when all the DNN tasks are finished. The NoC platform is modified from the cycle-accurate NoC simulator in [37], which has a similar architecture to the widely used Garnet in Gem5 [2].

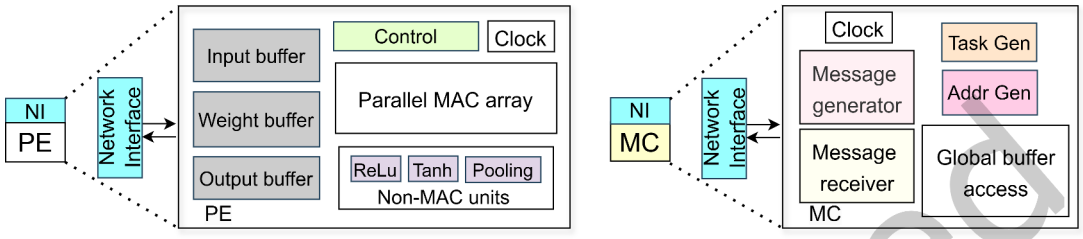

Fig. 3. Basic modeling of PE and MC.

Fig. 3 illustrates the modeling of PE and MC cores. Since NI is modeled with packetizing and de-packetizing components [3], PE and MC will only process messages instead of NoC-level packets. PE consists of buffers and computational units. The parallel MAC array is for common MAC operations in DNN layers, it takes input and weight from the local buffers and generates the result. The non-MAC units are in charge of other operations such as non-linear activation and max pooling. The output buffer stores any intermediate result, and PE will create a message to carry the final result back to MC. MC consists of two message blocks and a global buffer access block. The message receiver decodes the message from PE. Similarly, the message generator creates the response message to PE according to the request. The task and address generators are functional blocks designed to prepare neuron tasks and data addresses. Then MC can read and write the memory hierarchy through the global buffer access block.

3.3.3 Statistical output. Our simulator provides four output measurements to assess the performance of the user-defined NoC-DNN, including the DNN inference result, latency, traffic trace, and memory access.

(1) Inference result: It shows the final output for full model evaluation, which signifies the efficacy and accuracy of the NoC-DNN simulation.
(2) Inference latency: It outputs the end-to-end and layer-wised latency in clock cycles after simulation. This provides insights into the processing capability with the given NoC-DNN architecture.
(3) Traffic trace: The packet traffic trace file records the NoC communications, which helps identify potential bottlenecks. This provides the potential for other trace-based NoC simulation tools to reproduce the DNN inference task without running actual inference workloads.
(4) Memory access: This refers to the number of packets sent to and received from MC, which can be extracted from the trace file. It quantifies the utilization of the on-chip buffer.

These four statistical outputs contribute to a holistic assessment of the user-defined NoC-DNN accelerator, which helps perform statistical comparisons between different NoC-DNN configurations.

4 NoCDAS simulation workflow

Based on the proposed NoCDAS implementation, the simulator requires three steps to evaluate the target DNN model and NoC hardware specification. **Fig.** 4 shows the simulation workflow to get the final inference performance results.

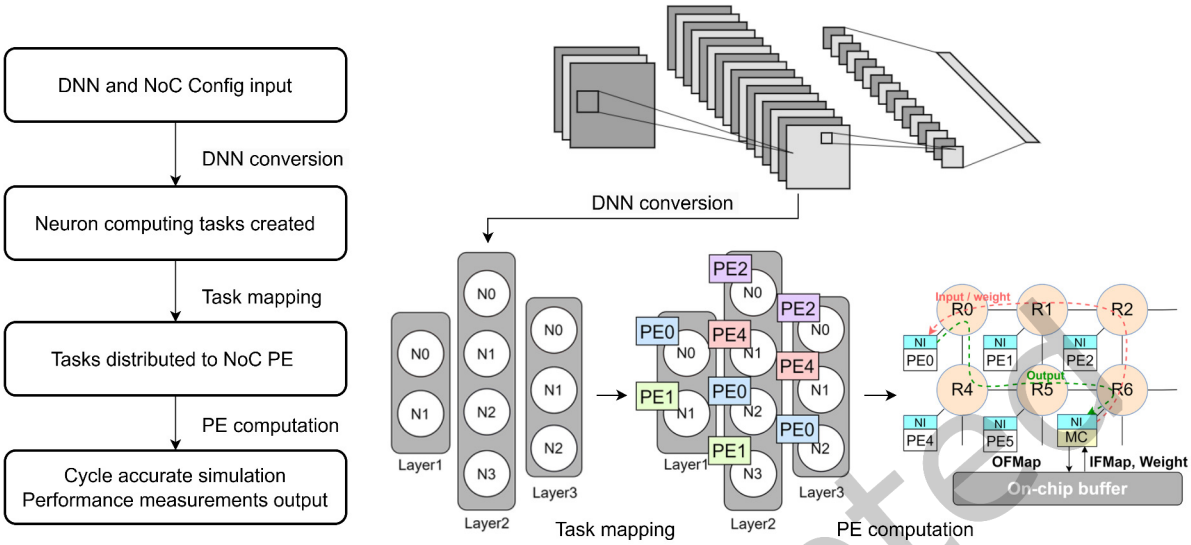

Fig. 4. NoC-DNN simulation flow.

4.1 DNN conversion process

The target DNN model usually comprises various types of layers, such as convolutional layers, pooling layers, and fully connected layers. In a fully connected layer (FC), one output is calculated with the MAC operations on all input neurons. For the pooling layer (Pool) and convolutional layer (Conv), one output only uses part of the inputs as a partially connected layer, whose size equals the shape of the pooling and convolution kernel, respectively.

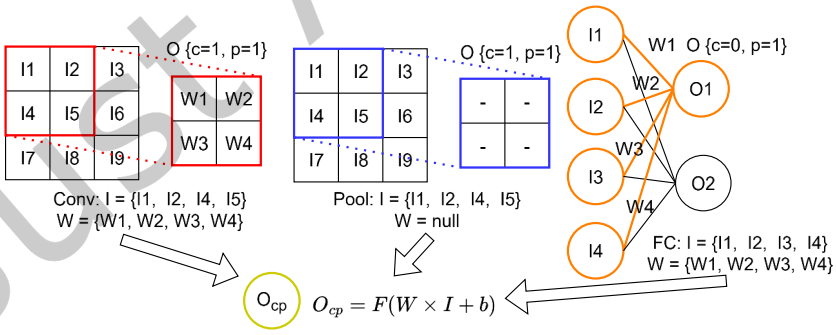

Fig. 5. DNN conversion process for different layers.

Though the layer types are different, their basic neuron computing expression is similar to the formula $O_{cp} = F(W \cdot I + b)$ shown in **Fig.** 5. In which W and I are the weight vector and input vector, b is the bias, and F is the non-linear activation function. O_{cp} represents the output value of the neuron N_{cp}, where c means its channel if applicable and p means its position in the output feature map (OFMap). For FC and Conv layers, the

size of vectors W and I are adjusted according to the layer and filter size. For Pool, a special F is adopted to fulfill the pooling operations such as *Max* and *Average* instead of the normal non-linear functions. In this case, W is a null vector, and the output is calculated directly by $F(I)$.

Given a DNN model, we need to break down its entire workload into smaller tasks to distribute them across PEs in the NoC. To standardize these tasks, we convert DNN models to the granularity of individual neurons. The task of each neuron is to get its output no matter what layer operation it has. As a result, our simulator can run any DNNs that are built with standard convolutional, dense, and pooling layers. During the DNN conversion process, layers in the DNN model are separated into neuron tasks, which ensure efficient parallelization over NoC hardware resources. The DNN conversion function is integrated with the task mapper interface.

4.2 Task mapping process

After the DNN conversion process, the created neuron tasks should be distributed to PEs in NoC. Since the number of PEs is limited compared with the converted neurons, especially for large-scale DNN models, we need a task mapping process to ensure the correct execution of these tasks.

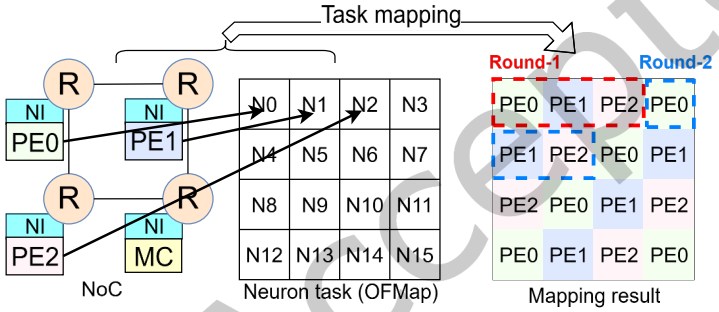

Fig. 6. Task mapping process over PE array.

The simulation follows a layer-completing order, so we separate the neurons layer by layer, and the task mapping process for each layer is called a mapping iteration. The basic mapping strategy considers a balanced workload distribution of neuron tasks over the whole NoC. The prevalent method is even mapping in Eyeriss V2 and CNN-Noxim [9, 11]. We adopt this method as it can be applied to any DNN tasks run on NoCDAS. Here we assign neuron tasks to PEs in the order of row-oriented mapping, until all neurons in this layer are attached to one of the PEs. **Fig.** 6 gives a mapping example of a 4×4 OFMap over three PEs. In this example, we assume at each time one PE can execute one neuron, while other mapped tasks are waiting in line. This is similar to the Direct-X mapping in CNN-NoXim but the scale of each mapped task is much smaller. Therefore, we don't need to allocate enormous local buffers inside each PE for the multi-core NoC.

The task mapper creates a mapping table for each layer and stores the mapped task IDs as a task list in its corresponding PE before the layer execution. Then PE knows the exact tasks it should perform during this mapping iteration. The task list updates after the layer is completed. Simultaneously, the on-chip global buffer updates the stored weights from the off-chip DRAM for the next layer.

Note that we support flexible configurations of mapping strategies to evaluate various data flows on the same NoC. This will change the task list in PEs, thus affecting the execution order of the converted DNN. In addition, the PE capability can be customized to process multiple neurons at a time to simulate some powerful and heavy PE designs.

4.3 PE computation flow

After the task mapping process, each PE has its dedicated neuron tasks to compute. We define a computation round as the time for processing one neuron. The proposed NoCDAS requires multiple rounds to complete the computation in one mapping iteration (or one layer). Since we need to simulate the network traffic accurately, the PE computation flow of one neuron is formulated in data packets. In each round, the PE activity to compute one neuron output can be divided into three steps:

(a) In the first step, PE sends a data request packet to MC, which involves the neuron task ID to fetch weight vector W, input vector I, bias b, and also the indicator for layer function F from the global buffer.
(b) In the second step, MC sends the requested data packet back to PE. MC accesses the on-chip buffer and prepares the data. The packet length depends on the amount of data as its payload. PE will start the computation after all the required information is received.
(c) In the third step, PE sends the result output packet to MC after its task is finished. The NoC configuration parameters define the computation latency of PE.

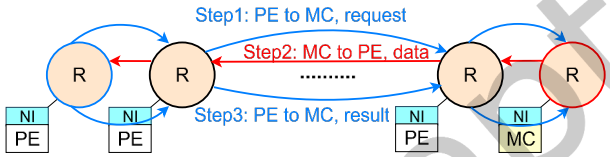

Fig. 7. PE computation flow in three steps.

Fig. 7 shows the traffic trace including these three steps between a PE and an MC. The destination MC of each PE also depends on the mapping strategy. In NoC-DNN, multiple PEs and MCs work on a large DNN model, generating complex traffic patterns. As a result, the proposed flit-level simulator NoCDAS can handle and record the traffic accurately for further evaluation.

In addition to the current PE computation flow, the NoCDAS simulator is not restricted to the small PE model. Users can attach more than one PE to the router to simulate a larger PE cluster that can process multiple neurons in one data transfer. Meanwhile, the task mapping and PE computation flow need adaptation to fit the large PE scenario.

5 Evaluation and exploration showcase

5.1 Experimental setup

To evaluate the proposed NoCDAS, we run three well-known DNN models on it, which are a small model LeNet [23], a medium model AlexNet [19] and a large model DarkNet [31]. Two groups of experiments are designed, including validating the simulator functionality and exploring NoC hardware configuration on running DNN models. The validation experiments will first examine the inference results of DNN with full model evaluation. Then the correctness of the hardware performance evaluation mode is validated by the inference latency using the same NoC configuration. During validation, we will use LeNet and AlexNet as our targets. After the functionality validation, we will explore various NoC-DNN design parameters and their affection on DNN inference latency using all three DNN models.

As a default setup, we initialize an 8×8 mesh NoC via NoC configuration. The NoC configuration parameters and their default values are summarized in **Tab.** 2. The network uses Virtual Channel (VC) routers, where several channels share a single physical channel to use the bandwidth efficiently. The protocol for transmission between VC routers involves flit-based wormhole transmission combined with credit-based flow control [12]. One data

packet is divided into smaller units called flits for fine-grained resource allocation. Each upstream router keeps track of the number of free flit buffers in each virtual channel downstream using a credit mechanism. Credits are decremented when the upstream router sends a flit and incremented when the downstream router frees the associated buffer. This ensures that the downstream router has sufficient virtual channel buffer space before flits are transmitted, preventing buffer overflow and thus no data loss.

We adopt a deterministic X-Y routing algorithm to avoid deadlock. The router frequency depends on the complexity of its logic for routing and resource allocation. In canonical 2D mesh-based multi-core System-on-Chip design, NoC can run much faster than processing cores [16]. We set the router frequency at 2 GHz as a common configuration [37], and the PE frequency is set to 200 MHz as in [24]. For the memory hierarchy, we simplify the memory access latency as MC read delay and MC data fetching delay. The latter is calculated by the bandwidth of the on-chip buffer. We set the MC transmission bandwidth to be 12.8 GB/s and the read latency is 5 ns (standard DDR3-1600 SDRAM) [17].

Table 2. NoC configuration parameters (default)

Topology	Mesh	Size	8×8
Flow control protocol	Virtual channel (VC)	Arbitration protocol	Round-Robin
VC per port	4	VC buffer depth	4 flits
Routing protocol	X-Y routing	Link width	256 bits
Link direction	Bi-direction	Data width	16 bits
Link latency	2 cycles	Router latency	1 cycle
MC bandwidth	6.4 Byte/cycle	MC access latency	1 cycle
Number of PEs	56	Number of MCs	8
Router frequency	2 GHz	PE frequency	200 MHz
PE capability	25 OPs / 1 non-linear activation per PE cycle		
Task mapping	Row-major mapping		

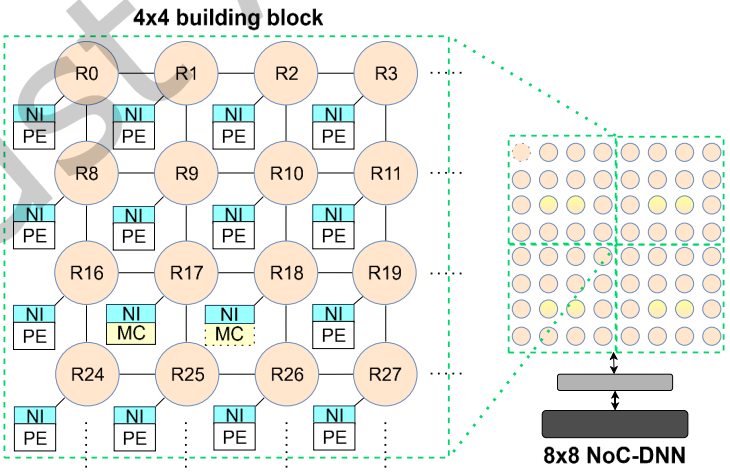

Fig. 8. Default NoC structure and its building block.

The default unit for latency measurement is one router clock cycle. For ease of simulation, we arrange router IDs in a row-major order starting with ID 0 and separate the whole network into four identical 4×4 building blocks. In total 8 MCs are deployed in the NoC, therefore 2 MCs are placed in each building block. **Fig.** 8 illustrates the building block on the left upper corner of the default NoC configuration, where two routers with ID 17 and 18 are connected to the MC. Each PE is assigned to the nearest MC within its block to minimize communication latency and balance the load across the MCs. This assignment is a design-time decision. In this default building block, PEs from the left two columns communicate with the MC at router R17 and the PEs from the right two columns communicate with the MC at router R18.

Based on the given configuration file, users can modify the configuration parameters to simulate different NoC-DNNs. The NoC mesh size can be arbitrary positive integers. The VC number, VC buffer depth, link width, NoC structural latency, and frequency can be changed according to users' design. The MC placement can be customized. There are eight sets of NoC sizes and MC positions predefined in the parameter file in NoCDAS. Users can add more NoC size and MC position setups by editing the MAC class in the NoCDAS program. We provide detailed guidance in the simulator README file.

The information of target DNN models is saved in DNN model structure files as part of the DNN configuration, and each line represents one layer in the model. The first line tells the width, height, and channel of the input layer. Then for Conv layers, the description includes the input channel, filter width, filter height, output channel, activation function, padding, and stride. For Pool layers, the parameters are the same as Conv layers, except for the activation function. For FC layers, only input size, output size, and activation function are required. The task mapper interface will perform DNN conversion and the converted parameters used in the simulation are listed in **Tab.** 3 (LeNet), **Tab.** 4 (AlexNet), and **Tab.** 5 (DarkNet). For DarkNet, we adopt the version with 19 convolutional layers, which is used as a backbone network for object detection model YOLOv2 [31]. The number of neurons counts the neuron tasks after DNN conversion. The number of OP counts the required operations for each layer, such as MAC operations for the convolutional layer, and Argmax comparisons for the max pooling layer. For example, the #OP in Conv1 layer of AlexNet is calculated by $(3 \times 11 \times 11) \times 290400 \approx 105.4$M. For ease of simulation, we assume that each operation takes the same time, and the PE capability is 25 OPs or 1 non-linear activation per PE cycle, as shown in **Tab.** 2. The number of rounds is calculated by $\lceil$#Neuron/#PE$\rceil$, which counts the neuron tasks mapped to each PE on average during one mapping iteration. In **Tab.** 3, **Tab.** 4, and **Tab.** 5 we calculate the number of rounds under the default NoC configuration.

Table 3. LeNet parameters for simulation (Input size $32 \times 32 \times 1$)

Layer	#Neuron	#Round	#OP	ACT func.
Conv1	4704	84	117.6k	ReLU
Pool1	1176	21	4.7k	-
Conv2	1600	29	240k	ReLU
Pool2	400	8	1.6k	-
FC1	120	3	48k	ReLU
FC2	84	2	10.1k	ReLU
FC3	10	1	840	Sigmoid

The configuration parameters for exploration are specified in the corresponding experiment subsections below. The inference latency is recorded in router cycles.

Table 4. AlexNet parameters for simulation (Input size $227 \times 227 \times 3$)

Layer	#Neuron	#Round	#OP
Conv1	290400	5186	105.4M
Pool1	69984	1250	0.63M
Conv2	186624	3333	447.9M
Pool2	43264	773	0.39M
Conv3	64896	1159	149.5M
Conv4	64896	1159	224.3M
Conv5	43264	773	149.5M
Pool3	9216	165	0.08M
FC1	4096	74	37.7M
FC2	4096	74	16.8M
FC3	10	1	409.6k

Table 5. DarkNet parameters for simulation (Input size $256 \times 256 \times 3$)

Layer	#Neuron	#Round	#OP	Layer	#Neuron	#Round	#OP
Conv1	2097152	37450	56.6M	Conv9	131072	2341	302M
Pool1	524288	9363	2.1M	Conv10	65536	1171	302M
Conv2	1048576	18725	302M	Conv11	131072	2341	302M
Pool2	262144	4682	1.05M	Conv12	65536	1171	302M
Conv3	524288	9363	302M	Conv13	131072	2341	302M
Conv4	262144	4682	302M	Pool5	32768	586	0.13M
Conv5	524288	9363	302M	Conv14	65536	1171	302M
Pool3	131072	2341	0.52M	Conv15	32768	586	302M
Conv6	262144	4682	302M	Conv16	65536	1171	302M
Conv7	131072	2341	302M	Conv17	32768	586	302M
Conv8	262144	4682	302M	Conv18	65536	1171	302M
Pool4	65536	1171	0.26M	Conv19	64000	1143	65.5M

5.2 Simulator validation

Firstly, we evaluate the inference results under full model execution (FE) mode to validate the proposed simulator NoCDAS. We use LeNet and AlexNet as the target DNN models. The input images are randomly chosen from MNIST for LeNet and resized CIFAR10 for AlexNet. The weights are trained in PyTorch and extracted by our script. The output feature map of each layer and the final inference results are printed from NoCDAS. Then we compare them with the PyTorch version and get the identical layer outputs and classification values. The correctness of NoCDAS can be ensured since the NoC packet delivery is correctly executed to get all the DNN neuron tasks done as we designed.

Then we validate our simulator's performance evaluation (RE) mode, which only takes the DNN structure file as DNN configuration. In RE mode, the weights and inputs are randomly generated for a quicker simulation flow without the DNN training process. Thus the inference results are not comparable for validation. In this case, we use the layer-wise latency results to validate that RE mode can give the same performance output as FE mode. We run the same DNN models with the default NoC setup in NoCDAS.

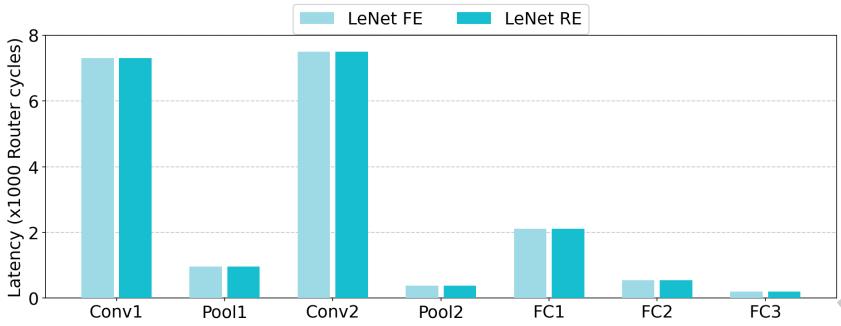

Fig. 9. LeNet layer-wise latency comparison between FE mode and RE mode.

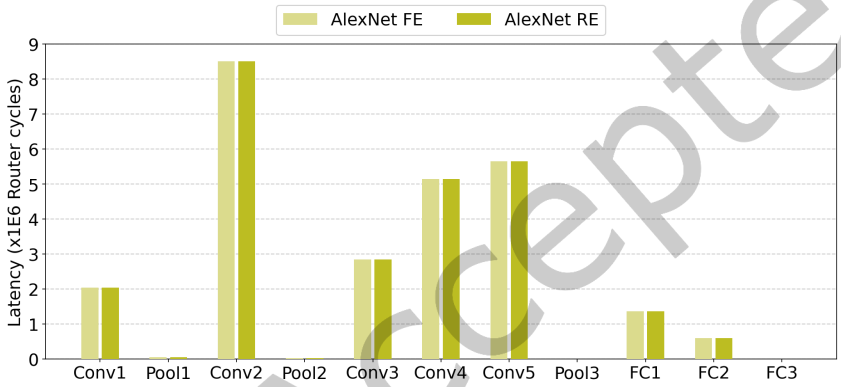

Fig. 10. AlexNet layer-wise latency comparison between FE mode and RE mode.

Fig. 9 and **Fig.** 10 plot the layer-wise latency of LeNet and AlexNet of two evaluation modes, in which we can observe identical inference latency in both modes. Thus the correctness of hardware performance measurement in RE mode is validated. A traffic trace file is generated along with the latency result, which records the packet ID, packet length, source, destination, creation time, and elimination time of every packet in NoC. There are 24282 packets created for LeNet and 2342238 packets created for AlexNet. Both of them are identical to the theoretical number, which is three times the number of neuron tasks after DNN conversion. The memory accesses are also extracted from this trace file.

Besides the function validation, we also measure the host execution time of our simulator. We run all three DNNs under RE mode using the default configuration as listed in **Tab.** 2. The simulator is run on a host desktop computer with Intel Core i5-10600 CPU and 16GB RAM. The runtime of LeNet is 1.195 seconds. AlexNet and DarkNet need 3258.110 seconds and 10098.528 seconds to complete, respectively. The average runtime per simulator clock cycle is around 0.077 milliseconds.

5.3 NoC hardware configuration exploration showcases

We present the NoC hardware exploration showcases upon the validation using the proposed NoCDAS. There are large design spaces for the NoC-DNN architectures, here we select three common design parameters as targets, including the NoC size, task mapping, and MC placement.

(1) NoC size: It defines the network size in NoC-DNN, which represents the scalability of the network. In the default setup, we use an 8×8 network with 4 building blocks, now we extend it to 12×12 and 16×16. In this case, they consist of 9 and 16 building blocks. We also evaluate it on a small NoC with only one 4×4 building block.
(2) Task mapping: It defines the neuron task mapping strategies. The default mapping is row-major mapping, which maps the neuron tasks along each row of PE. It is similar to the Direct-X mapping method [9] in CNN-Noxim. We also test other two mapping methods, column-major and random mapping. **Fig.** 11 gives the example task mapping outputs 16 neuron tasks on three PEs. The neuron task execution sequence in PE depends on the mapping result.
(3) MC placement: It defines the amount and position of MC in NoC. This will affect the data transmission distance between PE and MC, and change the NoC activity. In addition to the baseline setup, two new building blocks are designed for evaluation. The first design places 2 MCs on the edge of the block. The second design only reserves one MC at the center of the 4×4 block.

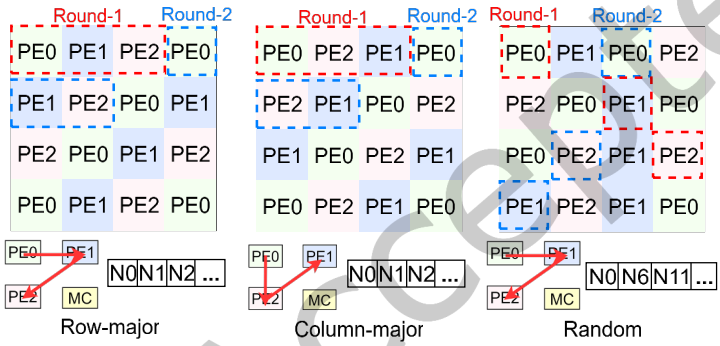

Fig. 11. Task mapping examples.

5.4 NoC hardware configuration exploration results

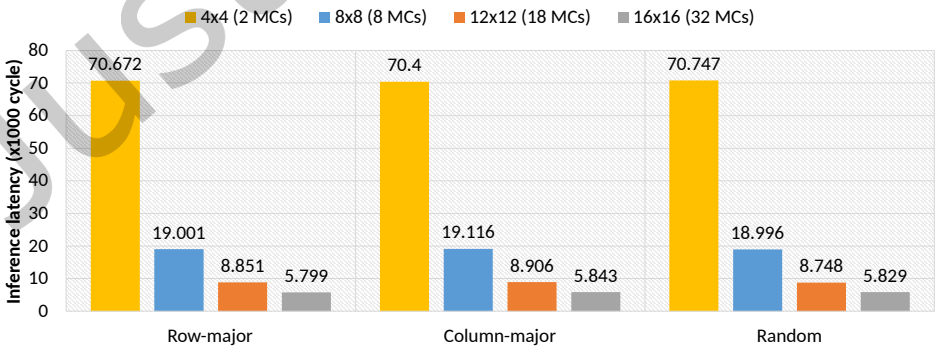

Fig. 12. LeNet inference latency. Design parameters: NoC size and task mapping.

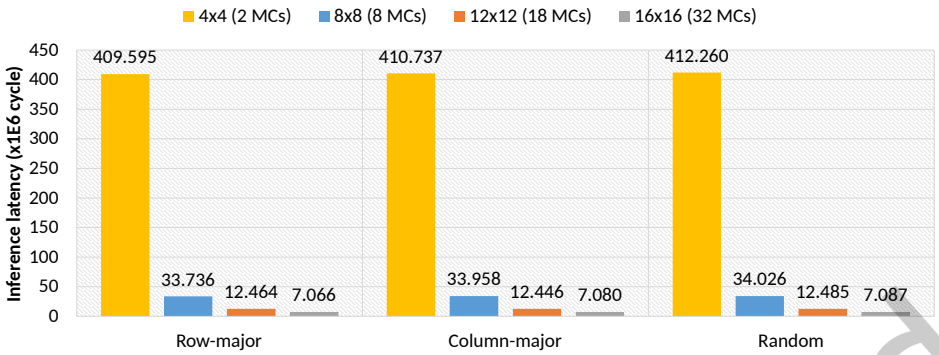

Fig. 13. AlexNet inference latency. Design parameters: NoC size and task mapping.

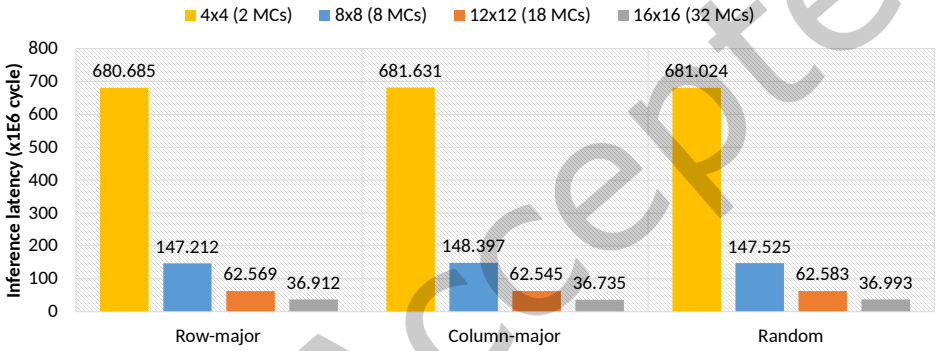

Fig. 14. DarkNet inference latency. Design parameters: NoC size and task mapping.

Fig. 12, **Fig.** 13, and **Fig.** 14 draw the inference latency results of three DNN models with two groups of design parameters, including four NoC sizes and three task mapping methods.

We first discuss the impact of NoC size using row-major mapping. With the quadratic growth of processing cores in NoC, the overall inference latency decreases exponentially in all cases. Since the NoC is configured to use the same building block structure, we can investigate the scalability efficiency by comparing the equivalent latency over one building block (1U), E_{lat} = Latency × #**U**. We normalize each E_{lat} based on the default 8×8 NoC. A smaller E_{lat} represents a better scalability efficiency on the target DNN.

Table 6. The equivalent latency of different NoC sizes, normalized according to the default 8x8 NoC (4U).

E_{lat}	4x4 (1U)	8x8 (4U)	12x12 (9U)	16x16 (16U)
LeNet	0.8256	1	1.0481	1.2208
AlexNet	3.0353	1	0.8313	0.8378
DarkNet	1.1560	1	0.9563	1.0487

All equivalent latency results are shown in **Tab.** 6. For LeNet, the 4×4 setup has the smallest E_{lat} over four NoC sizes. Since the LeNet doesn't have a tremendous number of tasks as modern DNNs, a small-sized NoC-DNN can

efficiently fit such a workload. For the medium DNN model AlexNet, the 12×12 NoC reaches the best scalability efficiency, and the E_{lat} of 16-by-16 is on the same level with around 0.6% difference. For the large DarkNet model. the 12×12 NoC has the best E_{lat} over four setups, and we can observe that 16×16 gives a worse E_{lat} compared with the default NoC. Therefore, a bigger NoC-DNN doesn't mean higher scalability efficiency. The proposed NoCDAS simulator can quickly find the most efficient scale of NoC to execute the target DNN models. In our showcase of NoC size exploration, a 4×4 NoC is suitable for LeNet and a 12×12 NoC can be chosen for AlexNet and DarkNet.

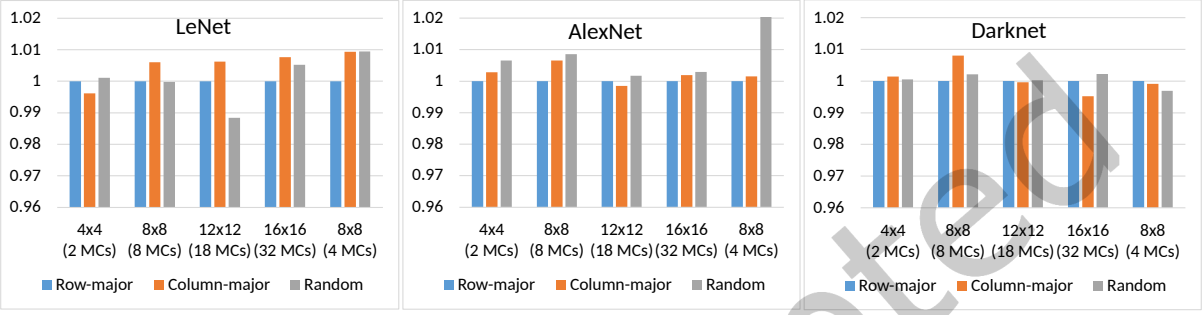

Fig. 15. Inference latency comparison of three task mapping methods, normalized to row-major mapping.

Then we evaluate the impact of task mapping strategies in NoC-DNN. **Fig.** 15 shows the normalized inference latency of three mapping methods over five NoC configurations based on the row-major mapping results. We can observe that the differences between them are not obvious. Since the computation tasks are designed to be evenly distributed to all available PEs in the evaluated mapping methods, PEs have the same amount of neuron tasks for each mapping iteration. Thus they have similar end-to-end latency. The differences happen mainly in the tail round, where some of the PEs are idle since the remaining tasks are less than the number of available PEs. In this case, the occupied PE will change according to task mapping, which would affect the latency in that round. For LeNet, row-major and column-major mapping are similar in all cases, since the workload distribution is similar in their tail round. For random mapping, only the 12×12 NoC shows a larger gap, which happens mainly in layer Conv2. The possible reason is that the 88 neuron tasks in this tail round got a better PE positioning in NoC which shortened the data path. For AlexNet, the 8×8 (4 MCs) NoC has the largest gap between the random mapping and the other two mappings. This happens mostly because of Conv2 and Conv3 layers as they have medium-sized tail rounds, where the differences in task mapping can be exposed. For DarkNet, the latency among different mapping is not obvious. Since the number of rounds in each convolution layer is huge, the latency difference of the tail round is concealed.

The neuron workloads of each PE using the three mapping strategies in this showcase are balanced. More complex mapping algorithms that consider uneven neuron task distribution have the potential to optimize the inference latency further. NoCDAS can be used to evaluate them efficiently.

Finally, the MC placement is evaluated by three different building blocks with the same NoC size of 8×8. Besides the default MC placement (8 MCs, center), we use two alternative designs. The first design (8 MCs, edge) still employs 2 MCs in each building block, but we place them on the edge. Similar to the example in **Fig.** 8, routers R8 and R16 are connected to the MC in this case. PEs in the top two rows are assigned to MC at router R8, and those in the bottom two rows are assigned to MC at router R16. The second design (4 MCs) uses the single-MC building block, where the router R18 is connected to the MC. With this change, the data transmission inside the building block becomes all-to-one traffic. The single-MC placement will worsen the performance compared to the other dual-MC scenarios due to the heavier MC accesses and longer communication latency.

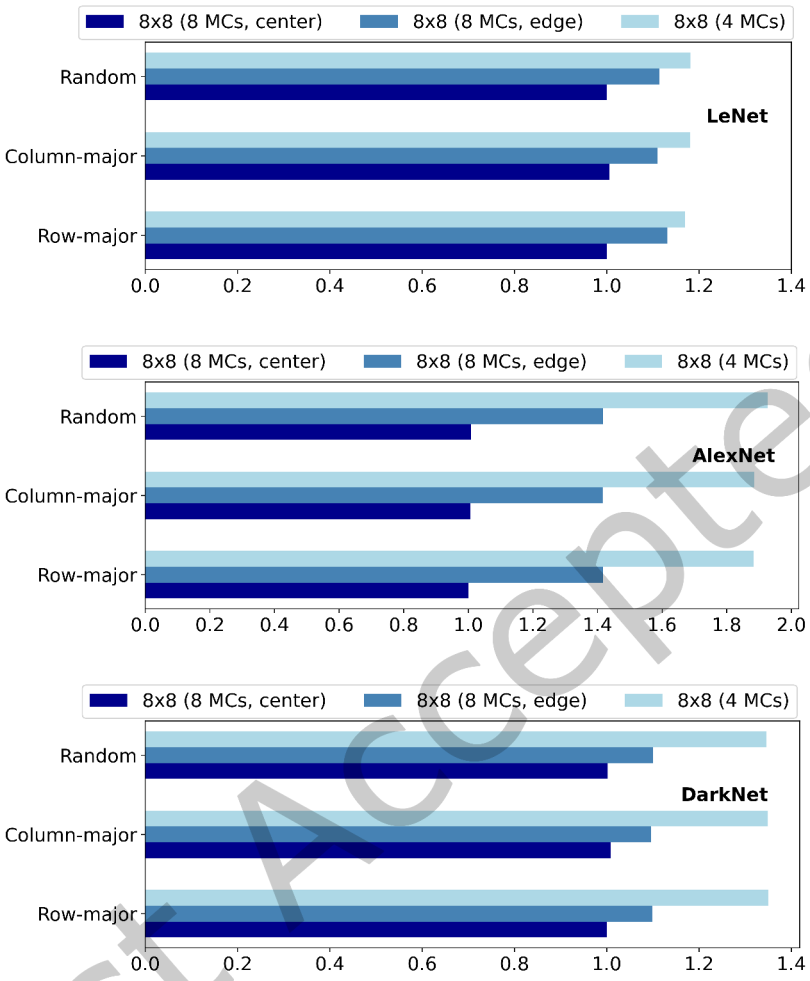

Fig. 16. LeNet, AlexNet, and DarkNet inference latency, normalized according to the default setup. Design parameter: MC placement.

Fig. 16 plots the inference latency of the DNN targets under these three MC placements. Though the 4-MC placement has more PE cores, it requires on average 1.18×, 1.90×, and 1.35× time to execute LeNet, AlexNet, and DarkNet compared with the default 8-MC placements. We can observe that MC becomes the bottleneck that limits the processing efficiency in such NoC-DNN configurations. Considering the two 8-MC cases, the design of edge placement needs 1.12×, 1.42×, and 1.10× inference latency on DNN models compared to the design with MCs placed at the center. This is mainly caused by the enlarged communication distance, with an average of 2.3 hops between PEs and MCs in the 8-MC edge placement scenario, compared to 1.7 hops in the default placement. By varying the MC placements, NoCDAS provides a rapid exploration of PE and MC resource allocation.

5.5 Comparison with DNNoC-sim

In this section, we compare the proposed NoCDAS with the DNNoC-sim to show the design parameter exploration ability differences between the two simulators. Since the computation flow and hardware assumption are different in the two simulators, the latency result is not directly comparable. Hence we evaluate them regarding the scalability exploration. By changing the NoC size parameter, one can analyze the simulator output to deploy the proper amount of NoC hardware resources for a certain DNN target.

For DNNoC-sim, we adopt the LeNet and VGG16 inference latency results on mesh-based NoCs from [6], which represent the small and large DNN workloads. We transform the time unit from second to router cycle according to their system frequency setup. Since different group sizes for partitioning the DNN models are reported for DNNoC-sim, we select the smallest one in both scenarios, which are 216 neurons per group for LeNet and 200 thousand neurons per group for VGG16. For NoCDAS, we change the NoC size and keep other NoC configuration parameters in default to run the same DNN models.

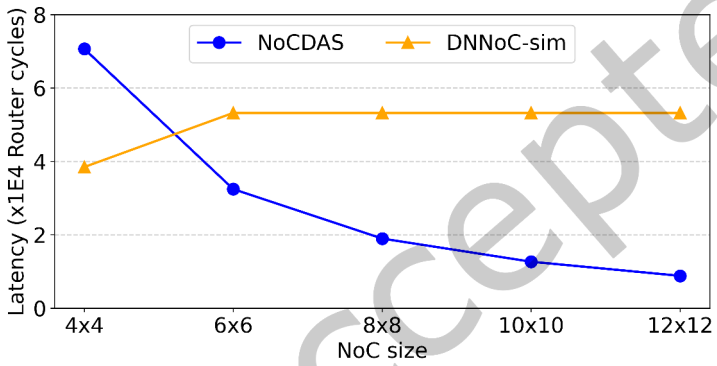

Fig. 17. LeNet end-to-end inference latency when varying NoC sizes in DNNoC-sim and NoCDAS.

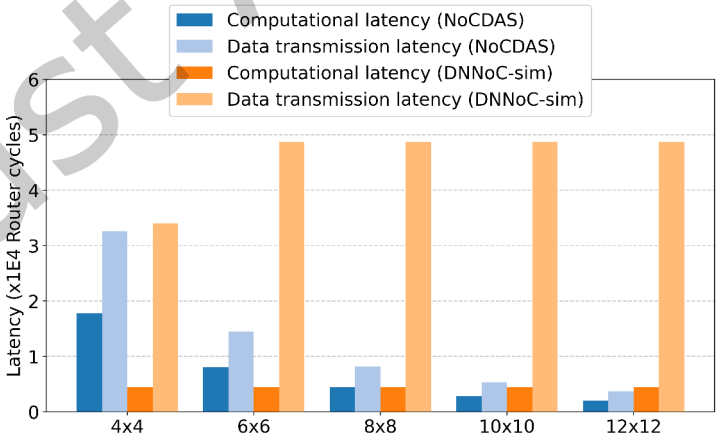

Fig. 18. LeNet inference latency breakdown when varying NoC sizes in DNNoC-sim and NoCDAS.

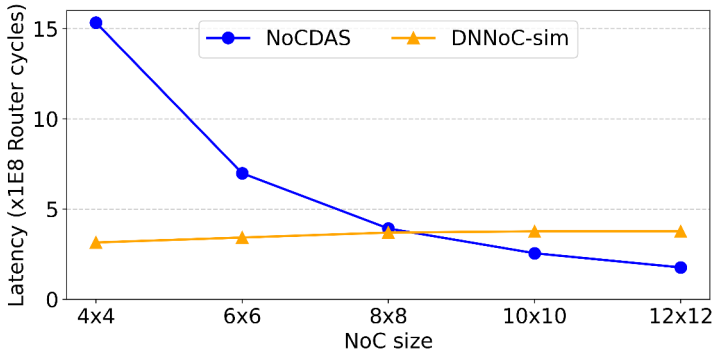

Fig. 19. VGG16 end-to-end inference latency when varying NoC sizes in DNNoC-sim and NoCDAS.

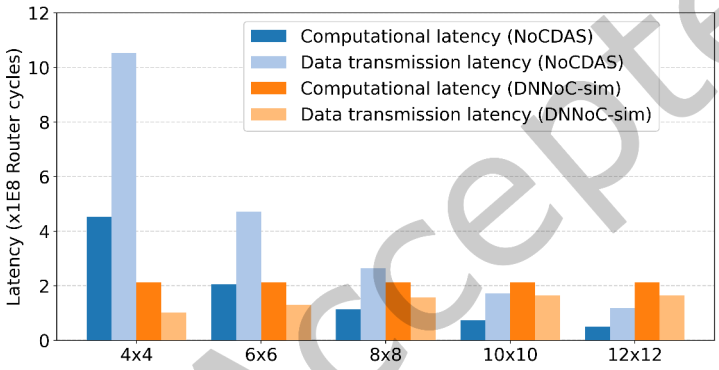

Fig. 20. VGG16 inference latency breakdown when varying NoC sizes in DNNoC-sim and NoCDAS.

Fig. 17 and **Fig.** 19 plot the inference latency results of LeNet and VGG16 in two simulators, where the evaluated five NoC sizes range from 4×4 to 12×12. Note that for 6×6 and 10×10 NoC, the default building block in NoCDAS cannot directly cover their area, so we assign 5 and 13 MCs respectively to keep the proportion of MC according to NoC size. In addition, the latency breakdown of LeNet and VGG16 are plotted separately in **Fig.** 18 and **Fig.** 20. For NoCDAS, the computational latency records the calculation time in PE. The data transmission latency involves the NoC communication time and the memory access time. Since the neuron tasks are evenly mapped over all PEs in hundreds of computation rounds, we count the latency on each neuron task and then average among all PEs in NoC to represent the actual workload on each PE.

For DNNoC-sim, the memory access latency is not included in its performance evaluation results, therefore we exclude this part from the shown data transmission latency. The computational latency in DNNoC-sim analysis is the sum of the PE computation time for each layer in each mapping iteration. Under their group setting, the LeNet model can be finished in two mapping iterations for the 4×4 NoC, and only one mapping iteration for other NoC sizes. For the VGG16 model, though it requires five mapping iterations for the 4×4 NoC, the largest layer can still be mapped to this NoC at once. Therefore, their computational latency remains the same when varying the NoC size.

In NoCDAS, the inference latency is inversely proportional to the NoC size, which indicates the better parallelism of increased processing cores in the network. However, in DNNoC-sim, the results are different. For

LeNet, the inference latency first increases and then becomes saturated after the NoC size reaches 6×6. For VGG16, the inference latency increases when the NoC size becomes larger and saturates at 10×10. The possible reason is the limited network utilization, which arises from the impractical assumption of large PE size and the ignored communication overhead from PE to memory in DNNoC-sim. When the neuron tasks in adjacent layers cannot be mapped simultaneously, the temporary OFMap results are written directly to the memory by PEs and later read by the corresponding PEs in the next mapping iteration. There is less traffic in the NoC in this situation. Therefore the data transmission time in LeNet is shorter in the 4×4 NoC as it is the only setup unable to map LeNet onto the NoC in a single step. Similarly, for VGG16, the data transmission latency increases until the 10×10 NoC, as the two largest NoC sizes in our experiment can map VGG16 in one mapping iteration.

As NoC-DNN is designed to achieve parallel processing in PEs, the performance should be improved when the scale of NoC increases. However, the increased NoC size leads to poorer overall latency results in DNNoC-sim, which negatively impacts its reliability. In contrast, the proposed NoCDAS can simulate such scalability well in NoC-DNN, therefore giving designers the correct insight into the design parameter of accelerator size.

In summary, the presented experimental results validate the correctness of our simulator NoCDAS and exhibit its unique ability on NoC-DNN design parameters exploration over other NoC-DNN simulators. Three NoC configuration showcases provide guidance on selecting latency and cost-efficient NoC-DNNs.

6 Conclusion

In this paper, we present NoCDAS, a cycle-accurate NoC-based DNN accelerator simulator. We first compare the current NoC-DNN simulators and discuss their features and limitations. Then we present our simulation framework design and explain the simulation workflow in detail. The simulation results are validated on both FE and RE modes in NoCDAS over two DNN models LeNet and AlexNet. We showcase the NoC-DNN design exploration over three design parameters, including NoC size, task mapping, and MC placement. We run three DNN models encompassing LeNet, AlexNet, and DarkNet to represent light to heavy workloads on NoC-DNN. Through these demonstrations, the potential of our simulator to assist better NoC-DNN hardware designs is proved.

We believe the proposed NoCDAS can be useful for NoC-DNN design space exploration. Different optimization strategies and workloads can be evaluated as add-ons to our simulator, such as layer-pipelining, in-network processing, sparse DNNs, and transformers. In-depth explorations of design parameters can help improve the efficiency of NoC-DNN. Multicast and broadcast transmission can be added for the exploration of data reuse methods. More performance measurements like the power model can be integrated into our simulator. The parallelization of simulation is also important to reduce running time when simulating large-scale NoC-DNNs. These will be investigated in the future.

Acknowledgments

The research was supported in part by Vetenskapsrådet (Swedish Research Council) through the LearnPower project (2020-03494).

References

[1] Sergi Abadal, Albert Mestres, Raul Martinez, Eduard Alarcon, Albert Cabellos-Aparicio, and Raul Martinez. 2015. Multicast on-chip traffic analysis targeting manycore NoC design. In *23rd Euromicro International Conference on Parallel, Distributed, and Network-Based Processing*. 370–378.

[2] Niket Agarwal, Tushar Krishna, Li-Shiuan Peh, and Niraj K Jha. 2009. GARNET: A detailed on-chip network model inside a full-system simulator. In *Proceedings of IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. 33–42.

[3] Hoda Ahmadinejad, Fatemeh Refan, and Hessam S Sarjoughian. 2011. NoC simulation modeling in DEVS-suite. In *Proceedings of Symposium on Theory of Modeling & Simulation: DEVS Integrative M&S Symposium*. 134–139.

[4] Vincenzo Catania, Andrea Mineo, Salvatore Monteleone, Maurizio Palesi, and Davide Patti. 2015. Noxim: An open, extensible and cycle-accurate network on chip simulator. In *Proceedings of IEEE 26th International Conference on Application-specific Systems, Architectures and Processors*. 162–163.

[5] Kun-Chih Chen, Masoumeh Ebrahimi, Ting-Yi Wang, and Yuch-Chi Yang. 2019. NoC-based DNN accelerator: A future design paradigm. In *Proceedings of 13th IEEE/ACM international symposium on networks-on-chip*. 1–8.

[6] Kun-Chih Chen, Masoumeh Ebrahimi, Ting-Yi Wang, Yuch-Chi Yang, and Yuan-Hao Liao. 2020. A NoC-based simulator for design and evaluation of deep neural networks. *Microprocessors and Microsystems* 77 (2020), 103145.

[7] Kun-Chih Chen and Yi-Sheng Liao. 2022. A reconfigurable deep neural network on chip design with flexible convolutional operations. In *Proceedings of 15th IEEE/ACM International Workshop on Network on Chip Architectures (NoCArc)*. 1–5.

[8] Kun-Chih Chen and Ting-Yi Wang. 2018. NN-noxim: High-level cycle-accurate NoC-based neural networks simulator. In *Proceedings of 11th IEEE/ACM International workshop on network on chip architectures (NoCArc)*. 1–5.

[9] Kun-Chih Chen, Ting-Yi George Wang, and Yueh-Chi Andrew Yang. 2019. Cycle-accurate noc-based convolutional neural network simulator. In *Proceedings of IEEE International Conference on Omni-Layer Intelligent Systems*. 199–204.

[10] Yu-Hsin Chen, Tushar Krishna, Joel S Emer, and Vivienne Sze. 2016. Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks. *IEEE journal of solid-state circuits* 52, 1 (2016), 127–138.

[11] Yu-Hsin Chen, Tien-Ju Yang, Joel Emer, and Vivienne Sze. 2019. Eyeriss v2: A flexible accelerator for emerging deep neural networks on mobile devices. *IEEE Journal on Emerging and Selected Topics in Circuits and Systems* 9, 2 (2019), 292–308.

[12] William James Dally and Brian Patrick Towles. 2004. Chapter 13 Buffered Flow Control. In *Principles and practices of interconnection networks*. Morgan Kaufmann Publishers Inc., 237–247.

[13] William James Dally and Brian Patrick Towles. 2004. Chapter 24 Simulation. In *Principles and practices of interconnection networks*. Morgan Kaufmann Publishers Inc., 473–475.

[14] Zidong Du, Robert Fasthuber, Tianshi Chen, Paolo Ienne, Ling Li, Tao Luo, Xiaobing Feng, Yunji Chen, and Olivier Temam. 2015. ShiDianNao: Shifting vision processing closer to the sensor. In *Proceedings of 42nd Annual International Symposium on Computer Architecture*. 92–104.

[15] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Deep residual learning for image recognition. In *Proceedings of IEEE conference on computer vision and pattern recognition*. 770–778.

[16] Jason Howard, Saurabh Dighe, Sriram R Vangal, Gregory Ruhl, Nitin Borkar, Shailendra Jain, Vasantha Erraguntla, Michael Konow, Michael Riepen, Matthias Gries, et al. 2010. A 48-core IA-32 processor in 45 nm CMOS using on-die message-passing and DVFS for performance and power scaling. *IEEE Journal of Solid-State Circuits* 46, 1 (2010), 173–183.

[17] J Janesky, ND Rizzo, D Houssameddine, R Whig, FB Mancoff, M DeHerrera, JJ Sun, M Schneider, HJ Chia, S Aggarwal, et al. 2013. Device performance in a fully functional 800MHz DDR3 spin torque magnetic random access memory. In *Proceedings of 5th IEEE International Memory Workshop*. 17–20.

[18] Liu Ke, Xin He, and Xuan Zhang. 2018. Nnest: Early-stage design space exploration tool for neural network inference accelerators. In *Proceedings of International Symposium on Low Power Electronics and Design*. 1–6.

[19] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. 2012. Imagenet classification with deep convolutional neural networks. *Advances in neural information processing systems* 25 (2012).

[20] Hyoukjun Kwon, Prasanth Chatarasi, Vivek Sarkar, Tushar Krishna, Michael Pellauer, and Angshuman Parashar. 2020. Maestro: A data-centric approach to understand reuse, performance, and hardware cost of DNN mappings. *IEEE Micro* 40, 3 (2020), 20–29.

[21] Hyoukjun Kwon, Ananda Samajdar, and Tushar Krishna. 2018. MAERI: Enabling flexible dataflow mapping over DNN accelerators via reconfigurable interconnects. *ACM SIGPLAN Notices* 53, 2 (2018), 461–475.

[22] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. 2015. Deep learning. *Nature* 521, 7553 (2015), 436–444.

[23] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. 1998. Gradient-based learning applied to document recognition. *Proc. IEEE* 86, 11 (1998), 2278–2324.

[24] Jinmook Lee, Changhyeon Kim, Sanghoon Kang, Dongjoo Shin, Sangyeob Kim, and Hoi-Jun Yoo. 2018. UNPU: An energy-efficient deep neural network accelerator with fully variable weight bit precision. *IEEE Journal of Solid-State Circuits* 54, 1 (2018), 173–185.

[25] Xiaoxiao Liu, Wei Wen, Xuehai Qian, Hai Li, and Yiran Chen. 2018. Neu-NoC: A high-efficient interconnection network for accelerated neuromorphic systems. In *Proceedings of 23rd Asia and South Pacific Design Automation Conference (ASP-DAC)*. 141–146.

[26] Francisco Muñoz-Martínez, José L Abellán, Manuel E Acacio, and Tushar Krishna. 2021. Stonne: Enabling cycle-level microarchitectural simulation for DNN inference accelerators. In *Proceedings of IEEE International Symposium on Workload Characterization (IISWC)*. 201–213.

[27] Seyed Morteza Nabavinejad, Mohammad Baharloo, Kun-Chih Chen, Maurizio Palesi, Tim Kogel, and Masoumeh Ebrahimi. 2020. An overview of efficient interconnection networks for deep neural network accelerators. *IEEE Journal on Emerging and Selected Topics in Circuits and Systems* 10 (2020), 268–282.

[28] Yiming Ouyang, Feiyang Tang, Chunlei Hu, Wu Zhou, and Qi Wang. 2021. MMNNN: A tree-based multicast mechanism for NoC-based deep neural network accelerators. *Microprocessors and Microsystems* 85 (2021), 104242.

[29] Angshuman Parashar, Priyanka Raina, Yakun Sophia Shao, Yu-Hsin Chen, Victor A Ying, Anurag Mukkara, Rangharajan Venkatesan, Brucek Khailany, Stephen W Keckler, and Joel Emer. 2019. Timeloop: A systematic approach to DNN accelerator evaluation. In *Proceedings of IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. 304–315.

[30] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. 2019. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems* 32 (2019).

[31] Joseph Redmon and Ali Farhadi. 2017. YOLO9000: better, faster, stronger. In *Proceedings of IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 7263–7271.

[32] Yakun Sophia Shao, Jason Clemons, Rangharajan Venkatesan, Brian Zimmer, Matthew Fojtik, Nan Jiang, Ben Keller, Alicia Klinefelter, Nathaniel Pinckney, Priyanka Raina, et al. 2019. Simba: Scaling deep-learning inference with multi-chip-module-based architecture. In *Proceedings of 52nd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 14–27.

[33] Yakun Sophia Shao, Sam Likun Xi, Vijayalakshmi Srinivasan, Gu-Yeon Wei, and David Brooks. 2016. Co-designing accelerators and SoC interfaces using gem5-Aladdin. In *Proceedings of 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 1–12.

[34] Vivienne Sze, Yu-Hsin Chen, Tien-Ju Yang, and Joel S Emer. 2017. Efficient processing of deep neural networks: A tutorial and survey. *Proc. IEEE* 105, 12 (2017), 2295–2329.

[35] Theocharis Theocharides, G Link, Narayanan Vijaykrishnan, MJ Invin, and Vamsi Srikantam. 2004. A generic reconfigurable neural network architecture as a network on chip. In *Proceedings of IEEE International SOC Conference*. 191–194.

[36] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).

[37] Boqian Wang and Zhonghai Lu. 2021. Flexible and efficient QoS provisioning in AXI4-based network-on-chip architecture. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 41, 5 (2021), 1523–1536.

[38] Sam Xi, Yuan Yao, Kshitij Bhardwaj, Paul Whatmough, Gu-Yeon Wei, and David Brooks. 2020. SMAUG: End-to-end full-stack simulation infrastructure for deep learning workloads. *ACM Transactions on Architecture and Code Optimization (TACO)* 17, 4 (2020), 1–26.

[39] Jiaqi Yang, Hao Zheng, and Ahmed Louri. 2023. Venus: A versatile deep neural network accelerator architecture design for multiple applications. In *Proceedings of 60th ACM/IEEE Design Automation Conference (DAC)*. 1–6.

[40] Shouyi Yin, Peng Ouyang, Shibin Tang, Fengbin Tu, Xiudong Li, Shixuan Zheng, Tianyi Lu, Jiangyuan Gu, Leibo Liu, and Shaojun Wei. 2017. A high energy efficient reconfigurable hybrid neural network processor for deep learning applications. *IEEE Journal of Solid-State Circuits* 53, 4 (2017), 968–982.

[41] Lingxiao Zhu, Wenjie Fan, Chenyang Dai, Shize Zhou, Yongqi Xue, Zhonghai Lu, Li Li, and Yuxiang Fu. 2023. A NoC-Based Spatial DNN Inference Accelerator with Memory-Friendly Dataflow. *IEEE Design & Test* 40, 6 (2023), 39–50.

[42] Kaiwei Zou, Ying Wang, Huawei Li, and Xiaowei Li. 2019. Learn-to-scale: Parallelizing deep learning inference on chip multiprocessor architecture. In *Proceedings of IEEE Design, Automation & Test in Europe Conference & Exhibition (DATE)*. 1172–1177.

Received 14 April 2024; revised 23 December 2024; accepted 14 March 2025